On the Use of Mutation in Injecting Test Order-Dependency

Sarra Habchi
University of Luxembourg
sarra.habchi@uni.lu

Maxime Cordy
University of Luxembourg
maxime.cordy@uni.lu

Mike Papadakis
University of Luxembourg
mike.papadakis@uni.lu

Yves Le Traon
University of Luxembourg
yves.letraon@uni.lu

arXiv:2104.07441v1 [cs.SE] 15 Apr 2021

Abstract*—*Background: **Test flakiness is identified as a major issue that compromises the regression testing process of complex software systems. Flaky tests manifest non-deterministic behaviour, send confusing signals to developers, and break their trust in test suites. Both industrial reports and research studies highlighted the negative impact of flakiness on software quality and developers' productivity. While researchers strive to devise solutions that could help developers addressing test flakiness, the elaboration and assessment of these solutions are hindered by the lack of datasets large enough to leverage learning techniques.** ***Aim:*** **To address this lack, we conduct an exploratory study that investigates a new mean for producing datasets of flaky tests.** ***Method:*** **We propose an approach that relies on program mutation to inject flakiness in software tests. In particular, we plan to delete *helper* statements from tests to make their outcomes order-dependent, *i.e.,* pass in certain running orders but fail in other orders. We intend to apply our mutation-based approach to a set of 14 Java projects to assess the effectiveness of test mutation in injecting order-dependency and generate a new dataset that could be used to study flakiness.**

***Index Terms*—Test flakiness, order-dependency, regression testing, mutation analysis.**

I. Introduction

Regression testing is a critical activity of modern software development. It enables simultaneous code development by validating program changes, *i.e.,* providing confidence that these changes do not break existing functionality. Nonetheless, this process can be hindered whenever non-deterministic behaviours are exhibited. In particular, flaky tests, which pass and fail for the same code version, send false alerts to developers and break their trust in the test suite. Developers waste considerable amounts of time investigating flaky test failures and this can slow down the continuous integration [1]. Besides, oft-repeated flaky failures may lead developers to disregard real test failures or even dispels them from writing tests. For these reasons, flakiness is widely discussed among software practitioners [2]–[5].

Several reports have shown that developers observe recurrently flaky tests and they struggle to address them. This struggle stems from the non-deterministic nature of flaky tests and the lack of suitable tools to cope with them. Specifically, developers are ill-equipped to distinguish flaky failures from real faults. Thus, they tend to rerun tests several times looking for persistent failures or passes. However, the rerun strategy is costly both computation and time-wise and its efficiency at identifying and reproducing flaky tests is questionable [6].

Responding to this need, researchers have started putting increasing amounts of efforts in understanding flaky tests and forming efficient techniques to detect them. Notably, the DeFlaker tool was proposed to detect flaky tests based on historical regression test executions [7]. Also, Lam *et al*. [8] proposed the iDFlakies tool that targets a different type of flakiness, tests for which the outcome changes depending on their order in the suite, *a.k.a.* order-dependent tests. Furthermore, researchers envision the usage of machine learning to address flakiness. Indeed, learning techniques are already used for predicting flaky tests statically [9]–[11]. Moreover, machine learning could assist in detecting, reproducing, and fixing flaky tests. Nevertheless, these applications are limited by the lack of large sets of flaky tests that could serve the purposes of learning and evaluating. Hence, there is a strong need for novel, large and diverse datasets of flaky tests.

Empirical studies rely on flakiness detection tools to build datasets from open source projects [7], [8]. For instance, a dataset was built using the DeFlaker tool, and it remains, to the best of our knowledge, the largest set with 1,874 flaky tests [7]. Yet, later studies showed that this set is hardly reproducible, which makes it difficult to use for learning fixes or root causes [10]. Nevertheless, it is hard to collect large, reproducible datasets from open source projects since flakiness appears in rapidly evolved (and tested) development environments. An additional root cause of flakiness regards the infrastructure used, which pose additional questions on the validity, responsibility and feasibility of the developed methods and their results.

In this paper, we present an exploratory study that aims to address the lack of test flakiness datasets, in general, and order-dependent tests in particular, from a different perspective. Instead of trying to mine test flakiness from projects, we propose FLAKER, an approach that injects order-dependency flakiness in software tests. FLAKER relies on program mutation to make tests order-dependent while it does not influence the test results. Our focus on order-dependent tests is motivated by the well-founded evidence about the sources and fixes of this type of flaky tests.

In particular, previous studies showed that order-dependency manifests through recurrent patterns of interactions between tests, such as *polluters*, *victims*, and *brittles* [8], [12]. Similarly,

Accepted at the MSR 2021 Registered Reports Track.

order-dependent tests can also be fixed thanks to *helper* statements that appear in *cleaners* and *state-setters* [13]. These *helpers* set or reset the shared state to lay the ground for a successful and stable test execution. The objective of our approach is to remove these *helpers* and make stable tests order-dependent. For this, we rely on program mutation, which is well-suited and widely used for these purposes.

We intend to evaluate FLAKER on 14 open-source Java projects that we collect from a public dataset. The objective of this evaluation is to answer the following research questions:

- **RQ1**: How effective is mutation in injecting order-dependent tests?

Objective: We aim to assess the suitability of test mutation for generating datasets of order-dependent tests.
Evaluation: We compare the number of order-dependent tests generated by mutation to the numbers of order-dependent tests that could be mined using detection tools like iDFlakies.

- **RQ2**: What are the characteristics of injected order-dependent tests?

Objective: We aim to characterise the injected order-dependent tests.
Evaluation: Following the terminology of previous research [13], we classify the generated tests in terms of *victims*, and *brittles*. We also investigate the nature of failures caused by the mutants, *e.g.,* Assertion errors or other exceptions.

- **RQ3**: What projects are more receptive to order-dependency injection?

Objective: We aim to identify and characterise testing systems that are prone to order-dependency injection.
Evaluation: We explore test suite properties that could explain the proneness to test order-dependency, *e.g.,* shared states, fixture usages, etc.

II. BACKGROUND

Order-dependent tests are a category of flaky tests that manifests a non-deterministic behaviour because of the execution order of the test suite. Such a situation may arise if the regression testing is performed using some form of test selection [14]. A test t is order-dependent if, for the same program and test suite, there exists a passing order O_p for which:

$$Run(t, O_p) = Pass$$

And a failing execution order O_f for which:

$$Run(t, O_f) = Fail$$

The problem of order-dependency has been particularly highlighted when projects updated their Java Development Kit version from 6 to 7. The new Java version modified the reflection implementation and therefore the order adopted by JUnit for running class tests evolved. Hence, test suites that used to pass correctly, started failing because of the new JUnit orders [15], [16]. Accordingly, developers realised that their suites are order-dependent.

As with other categories of flaky tests, order-dependent tests hinder regression testing as they waste development time and introduce uncertainty in the test suite. Besides these effects, order-dependent tests also impact the effectiveness of techniques that aspire to optimise regression testing. The study of Lam *et al.* [17] showed that order-dependent tests should be accounted for in test selection, prioritisation, and parallelisation techniques. The dependency between tests makes it difficult to reduce or rearrange the test suite without leading to non-deterministic failures.

Listing 1: Example of state polluters and cleaners with a victim test.

```
1
2 //Polluter
3 @Test
4 public void customConnectionFactory() {
5   handler = new RequestHandler() {...};
6   ConnectionFactory factory =
7     new ConnectionFactory(){
8     ... };
9   HttpRequest.setConnectionFactory(factory);
10  int code = get("http://not/a/real/url").code();
11  assertEquals(200, code);
12 }
13
14 //Cleaner
15 @Test
16 public void nullConnectionFactory() {
17  handler = new RequestHandler() {...};
18  HttpRequest.setConnectionFactory(null);
19  int code = get(url).code();
20  assertEquals(200, code);
21 }
22
23 //Victim
24 @Test
25 public void postWithNumericQueryParams() {
26  Map<> inputParams = new HashMap<>();
27  inputParams.put(1, 2);
28  inputParams.put(3, 4);
29  final Map<> outputParams = new HashMap<>();
30  handler = new RequestHandler() {...};
31  HttpRequest request = post(url,inputParams,false);
32  assertTrue(request.ok());
33  assertEquals("2", outputParams.get("1"));
34  assertEquals("4", outputParams.get("3"));
35 }
```

Previous studies showed that order-dependency is a result of interactions between different types of tests [12], [13], [18]. In the following, we define and explain these types of tests and their role in the passing and failing orders of flaky tests.

A. Polluters, victims, and cleaners

The study of Gyori *et al.* [12] identified *polluters* as a source of order-dependent tests. These tests are not order-dependent themselves but they pollute the state shared between tests and lead to the failure of tests that are executed after them, *a.k.a. victims*. Accordingly, a *victim* is a test that passes when run alone but fails when run with polluting tests. Following the formalisation of Shi *et al.* [13], a test is defined as a victim v, if there is a non-empty set of polluters P for which:

$$Run(v) = Pass$$
$$Run(P, v) = Fail$$

In the same vein, *cleaners* are tests that contain *helper* statements that clean the shared state and allow victims to pass. A set of tests C are considered cleaners if there exists a victim v and a set of polluters P for which:

$$Run(P, C, v) = Pass$$

Listing 1 shows an example of a test class from the ***HTTP Request*** project that contains a polluter, a cleaner, and a victim[1]. In this example, `customConnectionFactory()` pollutes the shared state by creating a `ConnectionFactory` and setting it in the static field of the `HttpRequest`, line 9. On the other hand, the test `nullConnectionFactory()` cleans that polluted state by resetting the factory of the `HttpRequest` to `null` in line 18. The test `postWithNumericQueryParams()` presents a victim because it uses the `HttpRequest` class without ensuring that its set properly. As a result, when the victim is run after the polluter, the `ConnectionFactory` is not null and the assertion in line 32 fails. However, when the same test is run alone or after the cleaner, the state is properly set and the assertions pass.

B. Brittles

Huo and Clause [18] identified *brittles* as another form of order-dependent tests. A *brittle* is a test that fails when run alone but passes when run after other tests. *Brittles* pass after other tests because they rely on other tests, called *state-setters*, to prepare the state required for their success. Specifically, a test b is considered as a *brittle* if there is a non-empty set of *state-setters* S that ensures that:

$$Run(b) = Fail$$
$$Run(S, b) = Pass$$

III. Execution Plan

In this section, we explain our execution plan by first presenting FLAKER, our approach for injecting order-dependent tests. Then, we present the evaluation protocol that we intend to follow in order to answer our research questions.

A. FLAKER

Figure 1 presents an overview of the FLAKER approach. FLAKER takes as input a Java program with its test suite and outputs a database of order-dependent tests. First, based on different rerun strategies, FLAKER selects stable tests from the program under analysis. Then, it mutates the selected tests to generate mutants that have potential order-dependency. Finally, it explores different running orders to evaluate and confirm the order-dependency in the mutant classes.

1) ***Step 1:*** *Select stable tests:* The objective of this step is to filter out tests that are already flaky before proceeding to flakiness injection. For this purpose, FLAKER extracts all tests from the program under analysis and checks their flakiness following a two-step process.

a) Detect non-order-dependent flaky tests: Non-order-dependent flaky tests represent tests that are non-deterministic because of general flakiness root causes, *e.g.,* concurrency and randomness, and not because of order-dependency. FLAKER identifies these tests by rerunning every test individually 100 times to check that its outcome is consistent. If a test result shifts in one of the execution, *i.e.,* $Pass \rightarrow Fail$ or $Fail \rightarrow Pass$, the test is considered flaky and we exclude it.

b) Detect order-dependent tests: We rely on randomised running orders to detect order-dependent tests. This strategy allows us to reveal dependencies between tests while avoiding the combinatorial explosion. Previous studies showed that randomised orders are the most effective for detecting dependency as they outperform reversal, exhaustive bounded, and dependence-aware bounded algorithms [19]. Since our tool targets Java programs, we only generate orders for tests that are in the same class and we do not consider the whole suite. We generate 20 random orders for each test class. This choice is based on the experiments of Lam *et al.* [8], which showed that 20 class orders are sufficient for detecting most flaky tests.

We run each generated order while keeping a record of test outcomes. Once all orders are run, we analyse the records to spot order-dependent tests, *i.e.,* tests that yielded different results over the executions. We exclude these tests because their non-determinism could influence the conduct of the upcoming steps.

2) ***Step 2:*** *Mutate tests:* The aim of this step is to generate test mutants that could lead to order-dependency. The study of Shi *et al.* [13] highlighted the presence of *helper* statements that reset the shared state and make tests order-independent. The objective of FLAKER is to remove these *helpers* and make tests order-dependent. For this, we relied on the mutation operator del_statement, which allows us to delete any statement from the test code. With this mutation, we can create mutant tests that are *brittles* or *victims*.

a) Generate brittles: Listing 2 presents an example of helpers found in the open-source project *Undertow*[2]. The example shows that the two tests `testIdleTimeout()` and `testCloseReason()` have an identical helper statement in lines 3 and 11 respectively. These *helpers* reset the state of the `Endpoint` and ensure that session connections are successful in both tests.

With mutation, we can remove the *helper* in line 11 and make `testCloseReason()` order-dependent. Indeed, without resetting the `Endpoint`, `testCloseReason()` would fail when run alone but would pass when run after the `testIdleTimeout()`. Consequently, `testCloseReason()` becomes a *brittle* and `testIdleTimeout()` becomes a *state-setter*.

Listing 2: Example of a helper statement.

```
1 @Test
2 public void testIdleTimeout() throws Exception {
3     Endpoint.reset();
4     Session session = deployment.connectToServer();
```

[1] https://github.com/kevinsawicki/http-request

[2] https://github.com/undertow-io/undertow

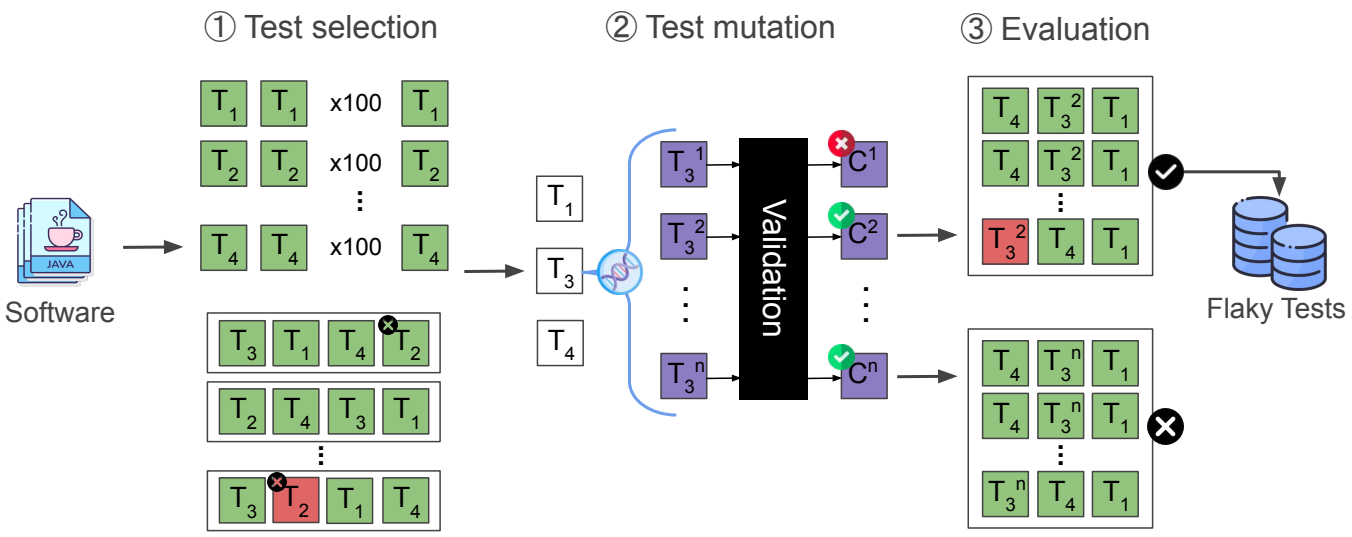


Fig. 1: Overview of the FLAKER approach.

```
5     Assert.assertEquals(..., Endpoint.message());
6     ...
7 }
8
9 @Test
10 public void testCloseReason() throws Exception {
11     Endpoint.reset();
12     MessageEndpoint.reset();
13     Session session = deployment.connectToServer();
14     session.close();
15     assertEquals("CLOSED",Endpoint.message());
16     ...
17 }
```

b) Generate victims: Listing 3 presents a modified example of potential *polluters* and *victims* in the *Hadoop* project[3]. In this example, the class has a test fixture method that runs once before all class tests. This method initialises the static variable `testDir` to prepare for test runs. On the other hand, `testWithStringAndConfForBuggyPath()` modifies this variable in line 11 and pollutes the shared state. However, `testAbsoluteGlob()` is protected from this pollution thanks to the *helper* in line 19, which reassures that this shared variable is set to the right value before using it in assertions. Hence, `testAbsoluteGlob()` is order-independent and it passes even when run after `testWithStringAndConfForBuggyPath()`.

With mutation, FLAKER can remove the helper from `testAbsoluteGlob()` and transform it into a *victim*. The test would pass when run alone because the initialisation sets the required state but would fail when run after `testWithStringAndConfForBuggyPath()`, which becomes a *polluter*.

Listing 3: Example of potential for a polluter and a victim.

```
1
2
3 @BeforeClass
4 public static void initialize() {
5   ...
6   testDir = new Path(System.getProperty(..)+"..");
7 }
8
9 @Test
10 public void testWithStringAndConfForBuggyPath() {
11   dirString = "file:///tmp";
12   testDir = new Path(dirString);
13   assertEquals("file:/tmp", testDir.toString());
14   checkPathData();
15 }
16
17 @Test
18 public void testAbsoluteGlob() {
19   testDir = new Path(System.getProperty(..)+"..");
20   PathData[] items = PathData.expandAsGlob(...);
21   assertEquals(sortedString(
22   testDir+"/d1/f1", testDir+"/d1/f1.1"),
23   sortedString());
24 }
```

As we cannot identify the *helper* statement beforehand, the mutation is applied to all test statements, except the assertions. That is, for each test with m non-assert statements, FLAKER generates m mutants. Each mutant test is reinserted in its mother class to form a mutant test class. If the mutant class does not compile, the mutant is considered invalid and therefore excluded from the analysis.

*3) **Step 3:** Evaluate mutants:* The aim of this step is to assess the impact of mutants on order-dependency. Algorithm 1 describes the process of identifying order-dependency in each mutant class.

First, we extract all the tests of the mutant class and we generate random running orders for them. Then, we run each of these orders and we save the execution results to keep track of orders that led to a test pass or failure. After executing all the generated orders, we verify if any of the class tests have

[3]https://github.com/apache/hadoop/

Algorithm 1: Detect order-dependent tests in mutant.

```
Inputs: mutantClass
Outputs: flakyTests[]
Procedure identifyFlakiness(mutantClass)
classTests[] = extractClassTests(mutantClass)
classOrders[] = generateTestOrders(mutantClass)
foreach classOrder O ∈ classOrders[] do
    passingTests[], failingTests[] = run(mutantClass, O)
    foreach test T ∈ classTests[] do
        if T ∈ passingTests[] then
            T.addPassingOrder(O)
        else
            T.addFailingOrder(O)
        end
    end
end
foreach test T ∈ classTests[] do
    if T.passingOrders ≠ ∅ and T.failingOrders ≠ ∅
     then
        flakyTests.addTest(T)
        if rerun(T) == Pass then
            T.isVictim = True
        else if rerun(T) == Fail then
            T.isBrittle = True
        else
            T.isNonOrderDependent = True
        end
    end
end
return flakyTests[]
```

both passing and failing orders. If so, this test is flaky and we need to classify it. For this purpose, we rerun the test 100 times alone to check if it is flaky regardless of the execution order. In such a case, the test is flaky but not order-dependent and we label it as such. If the test passes consistently through the reruns, this means that it is a *victim* of other polluting tests, which makes it fail when run with other tests. However, if the test fails when run alone, we consider it as a *brittle* that needs to run with other *state-setters* to pass.

B. Evaluation

Table I presents the metrics that we intend to compute after applying FLAKER. In the following, we explain how we leverage these metrics to answer our research questions.

1) ***RQ1: How effective is mutation in injecting order-dependent tests?:*** First, we compute the proportion of order-dependent mutants compared to the number of stable tests. This allows us to assess the capacity of mutation in injecting order-dependent tests. Also, to highlight the success rate of the mutation, we calculate the proportion of order-dependent mutants compared to all the generated and valid mutants.

To evaluate the suitability of the mutation approach for generating order-dependency datasets, we compare the numbers of generated order-dependent tests to the numbers of order-dependent tests that could be mined using other tools. Specifically, we compute the metrics **%OD tests** and **%OD classes** for the order-dependent mutants generated by FLAKER. Then, we compare these values with the proportions of **%OD tests** and **%OD classes** that could be detected using the tool IDFLAKIES [8]. These metrics can be retrieved from the results of previous studies, which applied IDFLAKIES on a set of Java projects. This comparison allows us to determine whether the mutation approach is a sound alternative for creating order-dependency datasets or not.

Furthermore, to assess the feasibility of the mutation approach, we report the average time consumed by FLAKER to perform the mutation and evaluate the order-dependency of mutants.

2) **RQ2*: What are the characteristics of injected order-dependent tests?:*** We categorise the order-dependent mutants into *brittles* and *victims* to explain the nature of order-dependent tests that we can inject with mutation. We also report the causes of failure, **%assertion** and **%exceptions** to show the impact of mutation on tests. These four metrics are also useful for the characterisation of the generated dataset and can be useful for studies that rely on it. On top of that, these metrics can help in comparing our injected mutants with real order-dependent tests that are observed in the wild.

3) **RQ3*: What projects are more receptive to order-dependency injection?:*** We aim to inspect the 14 projects under study and identify characteristics that could explain their level of proneness to flakiness injection. For this purpose, we analyse test suite properties, which could be related to order-dependency. For instance, we compute the size of test classes and analyse its impact on the success of flakiness injection. We also investigate the effect of shared resources, *e.g.,* static fields, which could be a sign of hidden states. Finally, we inspect the presence of test fixtures, *i.e.,* `setUp()` and `tearDown()` methods, which could protect tests against our mutation. To assess the effect of these metrics on the number of injected order-dependent mutants, we use the following statistical tests:

- Spearman's rank coefficient [20]: a non-parametric measure that assesses how well the relationship between two variables can be described using a monotonic function. We use this coefficient to assess the effect of class size and static fields on the number of injected order-dependent mutants. This measure does not require the normality of the variables and does not assess the linearity, hence its adequacy for our analysis.
- Mann-Whitney U test [21]: we use this test to check if the distributions of injected order-dependent tests are identical in the classes that have fixtures or not.
- Cliff's δ [22]: a non-parametric effect size measure, which is suitable for ordinal data. We use this measure to quantify the effect size of the presumed difference between the classes based on the presence of fixture.

It is worth noting that this question is exploratory and its analysis will depend on the injection results and the project properties. Hence, the axes presented here are only preliminary and other metrics can be added later.

TABLE I: Evaluation metrics.

	Metric
RQ1	**#stable tests**: The number of tests selected after filtering out tests that are already flaky in the first step of FLAKER.
	#mutants: The total number of mutants generated by FLAKER at step 2 (before validation).
	#valid mutants: The number of mutants that were successfully compiled after their reinsertion in the test class. This metric provides insights about the effects of mutation on the test correctness.
	#OD mutants: The number of mutants that show order-dependency at the evaluation step.
	%OD tests: The ratio between the number of order-dependent tests and the total number of tests in a project.
	%OD classes: The ratio between the number of order-dependent classes and the total number of test classes in a project.
	run time: The average execution time for FLAKER on different projects and test classes.
RQ2	**%brittles**: The percentage of order-dependent mutants that could not pass if run alone.
	%victims: The percentage of order-dependent mutants that could pass when run alone.
	%assertion: The percentage of order-dependent mutants that fail due to assertion errors.
	%exceptions:The percentage of order-dependent mutants that fail due to other exceptions or errors.
RQ3	**class size**: The number of tests in a test class.
	#static fields: The number of static fields in a test class.
	fixture: A boolean indicating the presence of fixture, *i.e.,* `setUp()` and `tearDown()` methods, in a test class.

IV. DATASET

In this section, we present the dataset that we aim to use in our study.

TABLE II: Evaluation set.

Project	*Link*
Alien4Cloud	https://github.com/alien4cloud/alien4cloud
DropWizard	https://github.com/dropwizard/dropwizard
Dubbo	https://github.com/apache/incubator-dubbo
Elastic-Job-Lite	https://github.com/elasticjob/elastic-job-lite
Fastjson	https://github.com/alibaba/fastjson
Hsac-Fitness-Fixture	https://github.com/fhoeben/hsac-fitnesse-fixtures
Junit-quickcheck	https://github.com/pholser/junit-quickcheck
Marine API	https://github.com/ktuukkan/marine-api
OpenPOJO	https://github.com/OpenPojo/openpojo
Spring-data-envers	https://github.com/spring-projects/spring-data-envers
Struts	https://github.com/apache/struts
Undertow	https://github.com/undertow-io/undertow
Wikidata-Toolkit	https://github.com/Wikidata/Wikidata-Toolkit
Wildfly	https://github.com/wildfly/wildfly

Table II shows the projects that we intend to use in our evaluation. These projects are collected from a public dataset[4] that was used in previous order-dependency studies. The original set of projects was first used to identify flaky tests with iDFlakies [8]. Later, Shi *et al*. used the tool iFixFlakies to patch order-dependent tests in 14 projects of this dataset [13]. In our study, we intend to focus on the same 14 projects to facilitate the early validation of our tooled approach. Specifically, if our mutation approach is capable of removing *helpers* and evaluating order-dependency accurately, it should be capable of reversing the fixes that were performed by iFixFlakies in these projects. On top of that, the former existence of order-dependency in these projects could be a favouring factor for our injection. For instance, these projects can already have shared states or polluting tests but their impact on tests is not visible yet because of the *helpers*. Our approach could reveal these dependencies by removing *helper* statements.

[4]https://sites.google.com/view/flakytestdataset

V. THREATS TO VALIDITY

A potential threat to our internal validity stems from our strategy for detecting order-dependency. We opted for randomised orders to detect order-dependent tests in both Step 1 and Step 3 of FLAKER. This strategy is not complete in that it does not explore all possible permutations of tests and it could miss some dependencies. Thus, it is possible that some order-dependent tests are not filtered at Step 1 and could interfere with our mutant evaluation. Similarly, some flaky tests could be missed in our mutant evaluation and hinder our study results. Nevertheless, achieving a complete order-dependency detection requires the exploration of all test permutations, which is impossible due to combinatorial explosion. The randomised algorithm showed its superiority compared to other incomplete algorithms like order reversal, exhaustive bounded search, and dependence-aware bounded search [19]. Besides, our configuration and choice of the number of reruns align with the positive results observed in previous studies on order-dependency detection [8].

Another possible threat arises from the use of mutation to generate a dataset of flaky tests. It is possible that the injected tests differ from the ones detected in the wild. To alleviate this threat, we report several metrics on the injected tests, *e.g.,* brittle, victim, and raised exception, to allow the comparison between our injected order-dependent tests and tests from other datasets.

VI. CONCLUSION

This paper presented our planned exploratory study on the injection of flakiness in software tests. The objective of this study is to evaluate the suitability of test mutation for generating datasets of order-dependent tests. To conduct this study, we proposed FLAKER, a tooled approach that removes *helper* statements from stable test suites to create order-dependency. This paper also presented the software projects and analysis protocol that we intend to adopt in our planned study.

REFERENCES

[1] M. Eck, M. Castelluccio, F. Palomba, and A. Bacchelli, "Understanding Flaky Tests: The Developer's Perspective," *arXiv*, pp. 830–840, 2019.

[2] J. Palmer, "Test flakiness – methods for identifying and dealing with flaky tests : Spotify engineering," https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/, November 2019, (Accessed on 01/12/2021).

[3] J. Listfield, "Google testing blog: Where do our flaky tests come from?" https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html, April 2017, (Accessed on 01/12/2021).

[4] J. Micco, "The State of Continuous Integration Testing Google," *Icst 2017*, 2017.

[5] M. contributors, "What's new in python 3.1 - python 3.9.1 documentation," https://developer.mozilla.org/en-US/docs/Mozilla/QA/Avoiding_intermittent_oranges, 2019, (Accessed on 01/20/2021).

[6] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, "Understanding reproducibility and characteristics of flaky tests through test reruns in java projects," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2020, pp. 403–413.

[7] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically Detecting Flaky Tests," in *Proceedings of the 40th International Conference on Software Engineering - ICSE '18*. New York, New York, USA: ACM Press, 2018, pp. 433–444. [Online]. Available: http://dl.acm.org/citation.cfm?doid=3180155.3180164

[8] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "IDFlakies: A framework for detecting and partially classifying flaky tests," *Proceedings - 2019 IEEE 12th International Conference on Software Testing, Verification and Validation, ICST 2019*, pp. 312–322, 2019.

[9] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a Bayesian Network Model for Predicting Flaky Automated Tests," *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, pp. 100–107, 2018.

[10] G. Pinto, B. Miranda, S. Dissanayake, M. D'Amorim, C. Treude, and A. Bertolino, "What is the Vocabulary of Flaky Tests?" *Proceedings - 2020 IEEE/ACM 17th International Conference on Mining Software Repositories, MSR 2020*, pp. 492–502, 2020.

[11] A. Bertolino, E. Cruciani, B. Miranda, and R. Verdecchia, "Know Your Neighbor: Fast Static Prediction of Test Flakiness," *Proceedings of the International Conference on Software Engineering (ICSE)*, 2020. [Online]. Available: https://ieeexplore.ieee.org

[12] A. Gyori, A. Shi, F. Hariri, and D. Marinov, "Reliable testing: detecting state-polluting tests to prevent test dependency," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis - ISSTA 2015*. New York, New York, USA: ACM Press, 2015, pp. 223–233. [Online]. Available: http://dl.acm.org/citation.cfm?doid=2771783.2771793

[13] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies : A Framework for Automatically Fixing Order-Dependent Flaky Tests," in *27th ACM Joint European Software Engineering Conference and Symposium on the Foundations ofSoftware Engineering (ESEC/FSE '19)*, 2019.

[14] C. Leong, A. Singh, M. Papadakis, Y. L. Traon, and J. Micco, "Assessing transition-based test selection algorithms at google," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2019, Montreal, QC, Canada, May 25-31, 2019*, H. Sharp and M. Whalen, Eds. IEEE / ACM, 2019, pp. 101–110. [Online]. Available: https://doi.org/10.1109/ICSE-SEIP.2019.00019

[15] Intellijava, "Java dev notes.: Junit and java 7." http://intellijava.blogspot.com/2012/05/junit-and-java-7.html, March 2012, (Accessed on 03/19/2021).

[16] B. Kunjummen, "all and sundry: Junit test method ordering," http://www.java-allandsundry.com/2013/01/junit-test-method-ordering.html, January 2013, (Accessed on 03/19/2021).

[17] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, "Dependent-test-aware regression testing techniques," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 298–311.

[18] C. Huo and J. Clause, "Improving oracle quality by detecting brittle assertions and unused inputs in tests," in *Proceedings of the 22Nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 621–631.

[19] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 385–396.

[20] J. H. Zar, "Spearman rank correlation," *Encyclopedia of biostatistics*, vol. 7, 2005.

[21] D. J. Sheskin, *Handbook of parametric and nonparametric statistical procedures*. crc Press, 2003.

[22] J. Romano, J. D. Kromrey, J. Coraggio, and J. Skowronek, "Appropriate statistics for ordinal level data: Should we really be using t-test and cohen'sd for evaluating group differences on the nsse and other surveys," in *annual meeting of the Florida Association of Institutional Research*, 2006, pp. 1–33.